

**INSTITUTO TECNOLÓGICO DE BUENOS AIRES
ESCUELA DE GESTIÓN Y TECNOLOGÍA**



Milo - Agente de IA para el Desarrollo Metacognitivo en Estudiantes

AUTORES: Santos BRAUN
Tomás FREIRE
Federico Inti GARCIA LAUBERER
Nicolás Andrés ROSSI

TUTORES: Federico José BOTTI
Alejo LEIVI

**TRABAJO FINAL PRESENTADO PARA OBTENER EL TÍTULO DE:
INGENIERO EN INFORMÁTICA**

**CIUDAD AUTÓNOMA DE BUENOS AIRES
Noviembre 2025**

Resumen

Milo es un agente de inteligencia artificial diseñado para fomentar la metacognición, la reflexión y el autoconocimiento en estudiantes. Su propósito es acompañar a cada alumno en el proceso de analizar su propio aprendizaje, su recorrido y, al mismo tiempo, descubrir aquello que realmente le interesa y apasiona. Milo actúa tanto en actividades propuestas por los docentes como en experiencias personales que el alumno decide registrar, formulando preguntas estocásticas y guiadas que estimulan la reflexión profunda.

El proyecto surge en respuesta a un problema estructural en la educación contemporánea: la brecha de preparación (*readiness gap*) entre lo que las escuelas miden y lo que los estudiantes necesitan para prosperar en la vida real. Hoy, 44 % de los empleadores afirman que los graduados carecen de habilidades como liderazgo, trabajo en equipo y resolución de problemas, aunque 95 % señala que son las más valoradas (Investopedia, 2024; Strauss, 2016). Asimismo, 89 % de los empleadores indica que las *soft skills* son difíciles de evaluar (LinkedIn, 2024), y solo 35 % de los egresados universitarios se siente preparado para el trabajo (Gallup, 2023).

Estos datos revelan que las instituciones educativas aún no cuentan con los medios adecuados para desarrollar, observar ni valorar las competencias humanas —como la autorregulación, la empatía o la capacidad reflexiva— que definen la preparación real para el futuro. Aunque muchas escuelas reconocen esta necesidad e incorporan marcos de aprendizaje socioemocional (SEL) y de competencias transversales, las herramientas actuales siguen centradas en evidencias académicas o cuantitativas, dejando de lado el acompañamiento personal del proceso de reflexión.

Milo aborda esta brecha desde una nueva perspectiva: utilizar la inteligencia artificial no para reemplazar el juicio humano, sino para facilitar espacios de metacognición y autodescubrimiento. A través de conversaciones guiadas, Milo ayuda a los estudiantes a identificar lo que les motiva, los contextos donde se desenvuelven mejor y las experiencias que despiertan su curiosidad. Paralelamente, brinda a las escuelas una nueva forma de visibilizar y valorar los intereses, fortalezas y pasiones de cada alumno, integrándolos como parte central del perfil de aprendizaje.

El proyecto propone el desarrollo de un prototipo funcional y de código abierto, integrable en entornos educativos formales y no formales, que promueva la reflexión como herramienta para la formación integral. De este modo, Milo busca contribuir a una educación más humana, personalizada y significativa, donde cada estudiante pueda reconocerse a sí mismo como aprendiz, creador y agente de su propio crecimiento.

1. Fases y cronograma del proyecto

1.1. Fase 1 | Análisis y diseño conceptual

Duración: Octubre 2025 – Noviembre 2025 (2 meses)

Objetivo: Definir el alcance, los requerimientos finales y la arquitectura técnica del sistema.

Actividades principales:

- Validación y cierre del documento de requisitos funcionales y no funcionales.
- Diseño del modelo de dominio, esquema de base de datos y arquitectura modular (STT, TTS, LLM Adapter, Policy Engine).
- Selección tecnológica (frameworks, proveedores de IA, stack de infraestructura).
- Diseño de *mockups* de interfaz (Figma / Miro).
- Configuración de entorno de desarrollo y repositorio (Bitbucket, CI/CD inicial).

Entregables:

- Documento SRS (versión final).
- Diagrama de arquitectura y flujos principales.
- Prototipo visual navegable.
- Backlog inicial priorizado.

1.2. Fase 2 | Desarrollo del MVP

Duración: Diciembre 2025 – Febrero 2026 (3 meses)

Objetivo: Implementar el núcleo funcional del sistema y los flujos básicos de reflexión por texto. Los docentes iniciaran el proceso reflexivo que luego servirá de guía para los estudiantes.

Actividades principales:

- Implementación del backend con endpoints principales (interacción, autenticación, registro).
- Creación del Policy Engine con familias de preguntas.
- Desarrollo del *frontend* básico (inicio de sesión, panel de reflexión, registro de confianza).
- Integración inicial con modelo LLM (OpenAI o similar).
- Persistencia de reflexiones y métricas básicas.
- Pruebas unitarias y de integración.

Entregables:

- MVP funcional (texto a texto).
- Documentación técnica de la API y arquitectura.

1.3. Fase 3 | Integración de voz, clasificación de reflexiones y prueba con usuarios

Duración: Marzo 2026 – Abril 2026 (2 meses)

Objetivo: Incorporar entrada y salida de voz y clasificación automática de reflexiones. Además se desarrollará una prueba del MVP con usuarios reales.

Actividades principales:

- Integración de módulo STT y TTS.
- Implementación de clasificación automática de reflexiones (descriptiva, analítica, crítica).
- Ajuste del Policy Engine para incorporar detección de emociones/frustración.
- Fast-forward de features o bugs que surjan de las pruebas con usuarios.

Entregables:

- Versión Beta del sistema con entrada/salida de voz.
- Clasificador metacognitivo operativo.

1.4. Fase 4 | Panel de métricas analíticas

Duración: Mayo 2026 (1 mes)

Objetivo: Implementar el sistema de métricas metacognitivas para los docentes.

Actividades principales:

- Desarrollo del *dashboard* docente.
- Integración con sistema de autenticación por roles (estudiante, docente, admin).
- Exportación de reportes PDF/CSV.

Entregables:

- Panel docente operativo.
- Reporte de métricas funcionales.

1.5. Fase 5 | Validación final y despliegue

Duración: Junio 2026 (1 mes)

Objetivo: Consolidar, probar e implementar la versión final del sistema.

Actividades principales:

- Despliegue del entorno de producción.
- Validación funcional y pedagógica del sistema completo.
- Corrección de errores críticos y afinamiento del rendimiento.
- Documentación final técnica y manual de usuario.
- Presentación de resultados y defensa del proyecto.

Entregables:

- Versión final estable de Milo (v1.0).
- Manual de instalación y operación.
- Informe de evaluación final y documentación académica.

2. Requerimientos del proyecto

2.1. Atributos de calidad

Availability: El sistema debe estar disponible de forma constante para su uso tanto en el aula como en entornos de aprendizaje autónomo. Los tiempos de inactividad promueven la desconfianza.

Performance: El sistema debe proporcionar interacciones rápidas para que el proceso de reflexión no resulte frustrante ni interrumpa el flujo de trabajo. Los estudiantes abandonarán la interacción si las demoras interrumpen el proceso.

Usability: El sistema debe ser intuitivo y facilitar su uso, reduciendo el esfuerzo mental necesario para su utilización.

Security: Proteger los datos de los estudiantes, garantizar la privacidad y cumplir con las políticas institucionales de compliance.

Scalability: El sistema debe estar preparado para soportar el crecimiento en el número de usuarios, cursos e instituciones que lo consuman.

Maintainability: El sistema deberá ser fácil de actualizar, ampliar y supervisar.

2.2. Métricas de calidad

M1. Tiempo de respuesta promedio El sistema será considerado satisfactorio si mantiene un tiempo de respuesta < 2 s para interacciones textuales y < 5 s para interacciones por voz, bajo condiciones normales de carga.

M2. Índice de usabilidad (SUS) La experiencia de usuario deberá alcanzar una puntuación ≥ 80 puntos en la escala System Usability Scale (SUS), equivalente a un nivel de satisfacción *excelente*.

M3. Porcentaje de disponibilidad mensual (Uptime) La plataforma deberá registrar una disponibilidad mínima del 99 % en promedio mensual, contabilizando tanto el acceso en el aula como fuera de ella.

M4. Cumplimiento de estándares de seguridad y privacidad Los análisis de seguridad (por ejemplo, mediante pruebas OWASP) no deberán reportar vulnerabilidades críticas, y el 100 % de los usuarios deberá haber otorgado consentimiento explícito antes de que sus datos sean procesados o almacenados.

M5. Escalabilidad y rendimiento bajo carga Pruebas de estrés progresivas en entorno cloud. Cumplimiento: desempeño dentro de umbrales de M1 con hasta 10.000 usuarios concurrentes.

M6. Mantenibilidad y calidad del código Análisis estático, revisiones de código y cobertura de tests. Cumplimiento: cobertura ≥ 80 %, complejidad ciclomática ≤ 10 y despliegues sin interrupciones.

2.3. Requisitos No Funcionales

RNF1. Performance

- RNF1.1 El sistema deberá responder a las interacciones textuales en un tiempo menor a 2 segundos, garantizando una experiencia fluida y sin interrupciones.
- RNF1.2 Las interacciones por voz (entrada y salida) no deberán superar los 5 segundos de procesamiento.
- RNF1.3 El *backend* deberá optimizar consultas, cacheo y uso de recursos para mantener un consumo eficiente bajo alta concurrencia.

RNF2. Usability

- RNF2.1 La interfaz deberá ser intuitiva y de fácil aprendizaje, permitiendo que un usuario nuevo comprenda sus funciones principales sin capacitación previa.
- RNF2.2 La navegación y disposición de los elementos deberán mantener una coherencia visual y funcional en todas las pantallas.
- RNF2.3 El diseño deberá ser adaptativo y responsivo, garantizando su correcto funcionamiento en computadoras, tablets y dispositivos móviles.
- RNF2.4 El sistema deberá ofrecer retroalimentación inmediata ante cada acción del usuario (confirmaciones, errores o progreso).

RNF3. Availability

- RNF3.1 Milo deberá mantener una disponibilidad mínima del 99% mensual, incluyendo horarios fuera del aula.
- RNF3.2 En caso de mantenimiento o fallas, el sistema deberá notificar a los usuarios anticipadamente, indicando tiempos estimados de recuperación.
- RNF3.3 Deberán existir mecanismos de recuperación automática ante errores menores para minimizar interrupciones.
- RNF3.4 Se realizarán copias de seguridad diarias automáticas de la base de datos, con retención mínima de 30 días.

RNF4. Security

- RNF4.1 Toda comunicación deberá estar cifrada mediante TLS 1.3, y los datos almacenados utilizarán AES-256.
- RNF4.2 Se implementará autenticación federada (OAuth2 / OIDC) y roles diferenciados (estudiante, docente, administrador).
- RNF4.3 El sistema solicitará consentimiento informado y explícito antes de almacenar o procesar información personal.

RNF4.4 Se realizarán pruebas de seguridad manuales basadas en los lineamientos de la OWASP Foundation, asegurando la ausencia de vulnerabilidades críticas.

RNF5. Scalability

RNF5.1 La arquitectura deberá permitir escalado horizontal para soportar incrementos en el número de usuarios o instituciones.

RNF5.2 El sistema deberá mantener un rendimiento estable con hasta 10 000 usuarios concurrentes en entorno *cloud*.

RNF5.3 Los componentes deberán ser modulares y desacoplados, permitiendo la extensión de servicios (STT, TTS, LLM Adapter, Policy Engine).

RNF6. Maintainability

RNF6.1 El código seguirá una arquitectura modular documentada, favoreciendo la legibilidad y separación de responsabilidades.

RNF6.2 La cobertura de pruebas unitarias deberá alcanzar al menos el 80 % del código.

RNF6.3 Las actualizaciones deberán poder realizarse sin interrumpir el funcionamiento del sistema.

RNF6.4 Se incluirán herramientas de monitoreo y *logging* centralizado para facilitar mantenimiento y diagnóstico.

3. Requisitos funcionales

3.1. RF1. Motor metacognitivo conversacional

RF1.1 El sistema deberá guiar al estudiante mediante preguntas que correspondan a las fases de planificación, monitoreo y evaluación del proceso metacognitivo. Incluyendo: definición de objetivos, autoevaluación, evaluación de estrategias y transferencia de conocimientos.

RF1.2 Variación aleatoria: El sistema deberá seleccionar y presentar las preguntas de forma aleatoria o con ligeras variaciones de redacción para evitar repeticiones y fomentar respuestas reflexivas.

RF1.3 Modo de respuesta: El sistema deberá evitar ofrecer respuestas directas o soluciones a las consultas del estudiante, reformulando las respuestas en forma de preguntas que promuevan la reflexión.

RF1.4 El sistema deberá adaptar el tipo de pregunta según la fase de la actividad y las señales del estudiante (nivel de confianza, tiempo de respuesta o frustración percibida).

3.2. RF2. Captura y representación de la reflexión

RF2.1 El sistema deberá permitir que los estudiantes ingresen sus reflexiones en formato de texto o voz.

RF2.2 En caso de entrada por voz, el sistema deberá transcribir automáticamente el audio a texto.

RF2.3 El sistema deberá permitir que el estudiante indique su nivel de confianza respecto de una tarea o concepto en una escala de 1 a 5.

RF2.4 El sistema deberá almacenar las reflexiones asociadas al usuario, a la actividad y a la fecha de creación.

RF2.5 El sistema deberá clasificar automáticamente cada reflexión según su nivel de profundidad (descriptiva, analítica o crítica).

3.3. RF3. Panel y funciones docentes

RF3.1 El sistema deberá permitir que los docentes creen actividades reflexivas asociadas a los objetivos y rúbricas del curso.

RF3.2 El sistema deberá mostrar al docente un panel de análisis con el nivel de profundidad de las reflexiones y la calibración entre confianza y desempeño de los estudiantes.

RF3.3 El sistema deberá permitir exportar reportes o portfolios con las reflexiones y métricas asociadas en formato PDF o CSV.

3.4. RF4. Gestión de usuarios y datos

RF4.1 El sistema deberá permitir el inicio de sesión mediante autenticación federada (OAuth2/OIDC) y diferenciar roles de usuario (estudiante, docente, administrador).

RF4.2 El sistema deberá almacenar toda la información de usuarios, reflexiones y actividades en una base de datos relacional segura.

3.5. RF5. Seguridad y privacidad

RF5.1 El sistema deberá analizar el contenido ingresado por los usuarios para detectar lenguaje inapropiado, tóxico o indicios de autolesión.

RF5.2 El sistema deberá solicitar y registrar el consentimiento informado del usuario antes de almacenar o procesar datos personales y reflexiones.

3.6. FR6. Análisis y mejora continua

RF6.1 El sistema deberá generar métricas automáticas de profundidad de reflexión, frecuencia de participación y niveles promedio de confianza.

RF6.2 El sistema deberá registrar métricas de uso (cantidad de sesiones, interacciones y tiempos de respuesta) para evaluar la adopción y rendimiento.

3.7. RF7. Accesibilidad y experiencia de usuario

RF7.1 El sistema deberá ofrecer sus respuestas en formato textual y de audio, con posibilidad de reproducir, pausar o repetir la salida.

4. Documentación de interfaces MVP

Milo contará con dos tipos de interfaces principales: las de usuario y las de sistema. Las primeras se centran en la experiencia de interacción de docentes y estudiantes, mientras que las segundas definen los mecanismos de comunicación entre los módulos internos y con plataformas externas.

En cuanto a las interfaces de usuario, el docente podrá crear actividades y experiencias de reflexión guiadas, configurando preguntas asociadas a los objetivos pedagógicos. También podrá visualizar los resultados individuales y el progreso de los estudiantes a través de un panel que integra indicadores de profundidad reflexiva, calibración y participación. Por su parte, el estudiante podrá iniciar una reflexión —ya sea asignada por el docente o propia— e interactuar con Milo mediante texto o voz. Durante esta interacción, el sistema generará preguntas guiadas y registrará las respuestas junto con el nivel de confianza indicado por el usuario. Una vez finalizada la sesión, la reflexión quedará almacenada y podrá visualizarse desde el panel personal o institucional. El flujo mostrado en el diagrama de esta sección sintetiza esta relación entre el docente, el estudiante y Milo, representando tanto la creación de experiencias guiadas como la interacción libre del usuario.

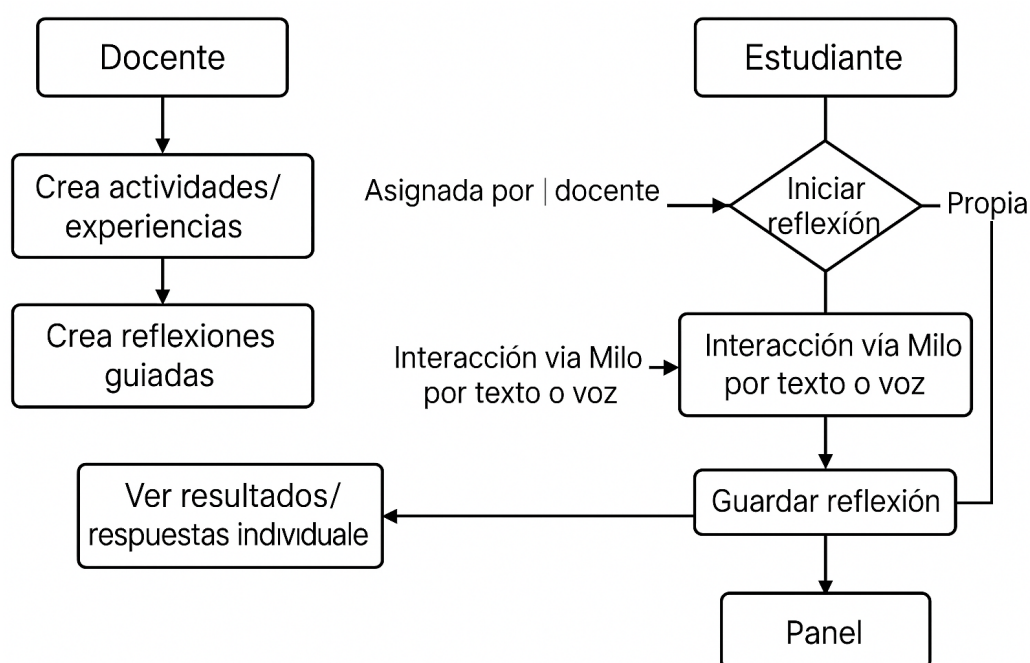


FIGURA 1: Diagrama de interfaces

5. Diseño de Alto Nivel

La arquitectura seleccionada es un modelo híbrido que combina dos sistemas monolíticos, frontend y backend, con servicios Serverless desacoplados que permiten delegar tareas asíncronas y computacionalmente costosas (como la transcripción de voz). En contraposición a una arquitectura de micro-servicios pura (escalable pero compleja de gestionar). Este diseño radica en una decisión estratégica de mantener la simplicidad que otorgan las arquitecturas monolíticas en estadios tempranos del proceso de desarrollo de software, manteniéndose el núcleo del sistema acoplado, pero dejando lugar para la flexibilidad con servicios especializados y altamente escalables.

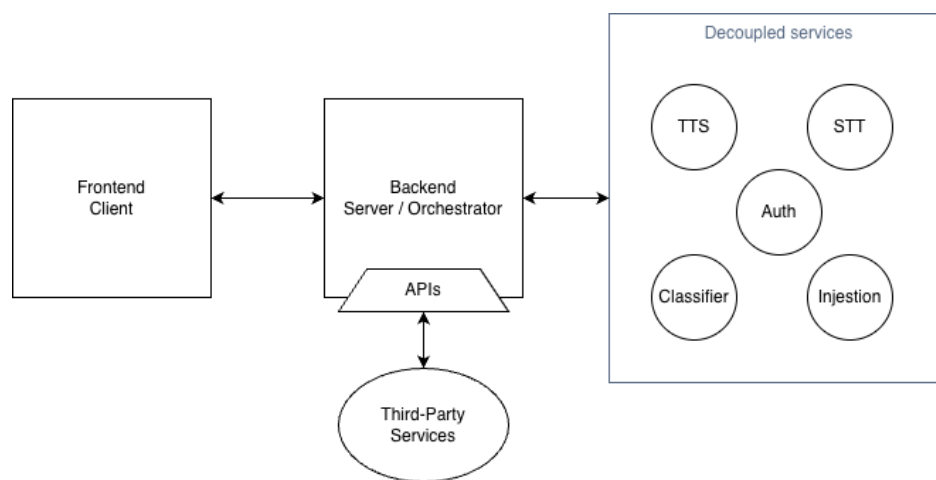


FIGURA 2: Diagrama de separación cliente-servidor y servicios

1. **Frontend:** Una SPA en React para toda la experiencia de usuario (estudiante y docente). Se despliega de forma independiente.
2. **Backend (Modular):** Un único servidor, usando ExpressJS con Typescript, para contener la lógica de negocio central, la orquestación del agente y la gestión de la comunicación con el cliente. Aunque se despliega como un servicio integral, su código estará organizado en módulos distintos para facilitar futuras migraciones.
3. **Servicios Serverless:** Un conjunto de funciones especializadas que se disparan a demanda y pueden ser fácilmente escalables.

5.1. Componentes del Sistema

Para favorecer la separación de responsabilidades y siguiendo las buenas prácticas actuales de la industria, la arquitectura del sistema se diseñó en base a cuatro capas interrelacionadas (Application, Orchestration, Model, y Data). Cada capa de la arquitectura cumple una función concreta, que abarca desde la gestión de los datos hasta la implementación de los modelos. Esta estructura permite identificar con mayor facilidad las dependencias, distribuir los recursos de manera eficiente y abordar los problemas de forma ordenada. (Cole Styrker, 2024)

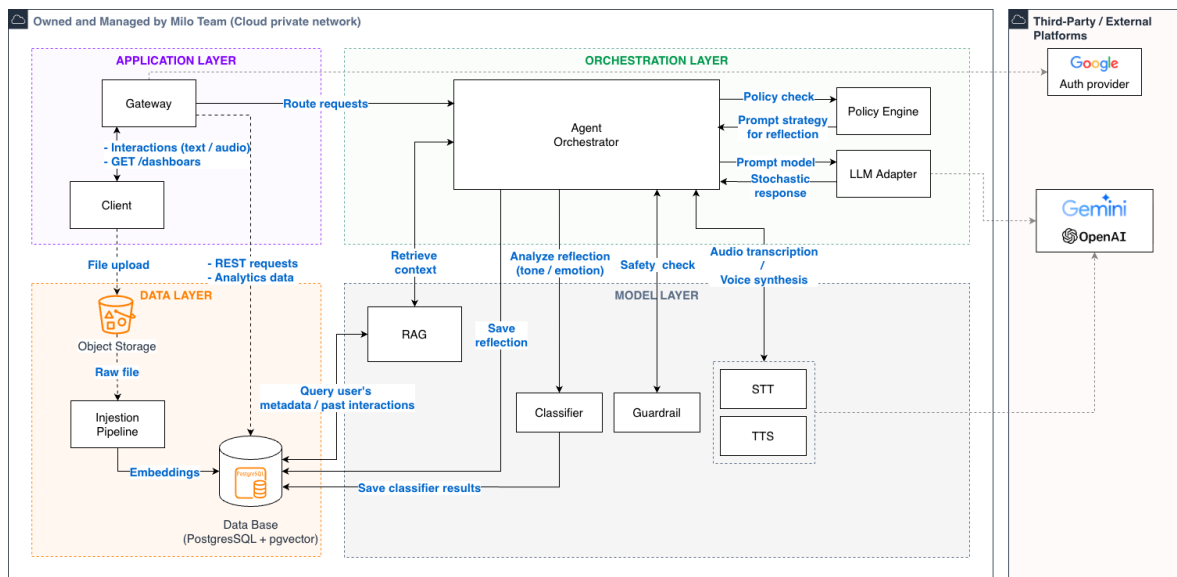


FIGURA 3: Diagrama de arquitectura de alto nivel

5.1.1. Capa de Aplicación

Responsable de toda la interacción con el usuario, lo conforman el cliente y el gateway. Desplegado en un servicio de hosting estático con una red de distribución de contenido (CDN).

Cliente (Client): Es la SPA que se ejecuta en el navegador o dispositivo del usuario. Proporciona la UI para la reflexión, los dashboards de docentes y la captura de entrada de datos.

Gateway: Es el punto de entrada único y seguro a la red de Milo. Gestiona la autenticación de usuarios (ej. contra el proveedor de Auth externo como Google) y enruta las peticiones entrantes.

- **Peticiones REST (HTTP):** Para operaciones sin estado como obtener datos del dashboard.
- **Interacciones (WebSocket):** Para la conversación de chat en tiempo real.

5.1.2. Capa de Orquestación

Se encarga de abstraer la complejidad de las diferentes subtarear que conforman a los distintos flujos de trabajo. Dentro de esta capa se encuentra toda la lógica de negocio. Agent Orchestrator: Es el componente central. Mantiene el estado de la interacción del usuario (ej. fase metacognitiva actual) y **ejecuta el bucle de conversación principal**. Enruta las solicitudes a los módulos internos y a los servicios externos.

1. Recibe el input
2. Consulta a los servicios de seguridad (safety checks)
3. Reúne el contexto (estado de la conversación, metadata del usuario)
4. Decide la estrategia pedagógica (policy engine)
5. Ejecuta el prompt y clasifica la respuesta (Large Language Model (LLM) Adapter, Classifier)
6. Devuelve la respuesta final al usuario

Policy Engine: Es un módulo interno. Este componente es la implementación técnica del "Motor metacognitivo". Es un sistema basado en reglas que lo rige que se diseñará como una máquina de estados. Los estados se corresponden directamente con las fases metacognitivas específicas. La función principal del motor es hacer cumplir los lineamientos metacognitivos como *'evitar ofrecer respuestas o soluciones directas'*. Esto se logra mediante el uso de "líneas de diálogo", que son conjuntos de reglas que interceptan y modifican tanto las consultas de los usuarios como las respuestas del LLM.

LLM Adapter: También es un módulo interno. El orquestador le envía un "Prompt model"(el prompt final, enriquecido por Retrieval-Augmented Generation (RAG) y el Policy Engine). El adaptador gestiona la comunicación con las APIs de LLM de terceros (Gemini, OpenAI, etc.). Este patrón de adaptador brinda modularidad , permitiendo cambiar o añadir proveedores de LLM sin alterar la lógica del orquestador.

5.1.3. Capa de Datos

En esta capa se gestiona la persistencia de los datos y los pipelines para poblar la base de datos híbrida. Todos los datos no estructurados deben convertirse a texto y luego a vectores de representación (embeddings).

Object Storage: Almacenamiento de objetos en la nube (AWS Simple Storage Service (S3)) para archivos binarios en crudo como audio, PDFs y otros que hayan sido subidos por los usuarios.

Ingestion Pipeline: Un servicio Serverless y asíncrono. Se activa automáticamente por medio de un evento, cuando un nuevo archivo llega al almacenamiento de objetos. Procesa el archivo para generar embeddings vectoriales que son guardados en la base de datos y serán utilizados como contexto para el LLM.

Data Base: En lugar de gestionar dos sistemas de datos separados (uno para metadatos y uno vectorial dedicado para embeddings), lo que crearía problemas de sincronización, com-

plejidad y costo, unificamos ambos con un diseño híbrido. Usamos PostgreSQL junto con su extensión de código abierto pgvector.

Este enfoque es superior por varias razones:

1. **Cumplimiento Relacional:** Todos los datos de usuario, roles, actividades y metadatos de reflexión se almacenan en tablas relacionales seguras.
2. **Almacenamiento Vectorial Nativo:** La misma tabla que almacena el texto de una reflexión y sus metadatos (como user, activity, confidence_level, entre otros) también almacena el embedding (vector) de ese texto.
3. **RAG Contextual Avanzado:** Este diseño híbrido permite al servicio RAG ejecutar consultas increíblemente potentes que los sistemas separados no pueden. Podemos combinar filtros relacionales (SQL WHERE) con búsqueda semántica (vectorial) en una sola consulta.

5.1.4. Capa de Modelo

Esta capa consiste en un ecosistema de servicios de IA especializados y desacoplados, que serán implementados como funciones Serverless (también podrían incluirse microservicios u otro módulos dedicados a resolver tareas específicas). El orquestador los invoca vía API.

Speech-to-Text (STT) / Text-to-Speech (TTS): Un conjunto de funciones que manejan la entrada/salida multimodal. El orquestador envía audio para transcripción y envía texto para síntesis de voz. **Guardrail:** Un servicio de clasificador síncrono. El orquestador le envía el texto del usuario para ejecutar un "Safety check" antes de procesarlo. Este servicio identifica lenguaje inapropiado, o riesgoso.

Classifier: Un servicio de clasificador que puede ser síncrono o asíncrono. Se usa para analizar la reflexión y guardar los resultados en la base de datos. Es un servicio fundamental para modelar el perfil y el progreso de los usuarios. Enriquece el contexto para mejorar las estrategias que provee la plataforma.

RAG (Retrieval-Augmented Generation): Un servicio invocado por el orquestador. Este servicio es el único que consulta la base de datos en busca de interacciones pasadas del usuario y metadatos relevantes que servirán como contexto para interactuar con el LLM.

6. Diagramas de secuencia para los flujos principales

6.1. Autenticación de usuario

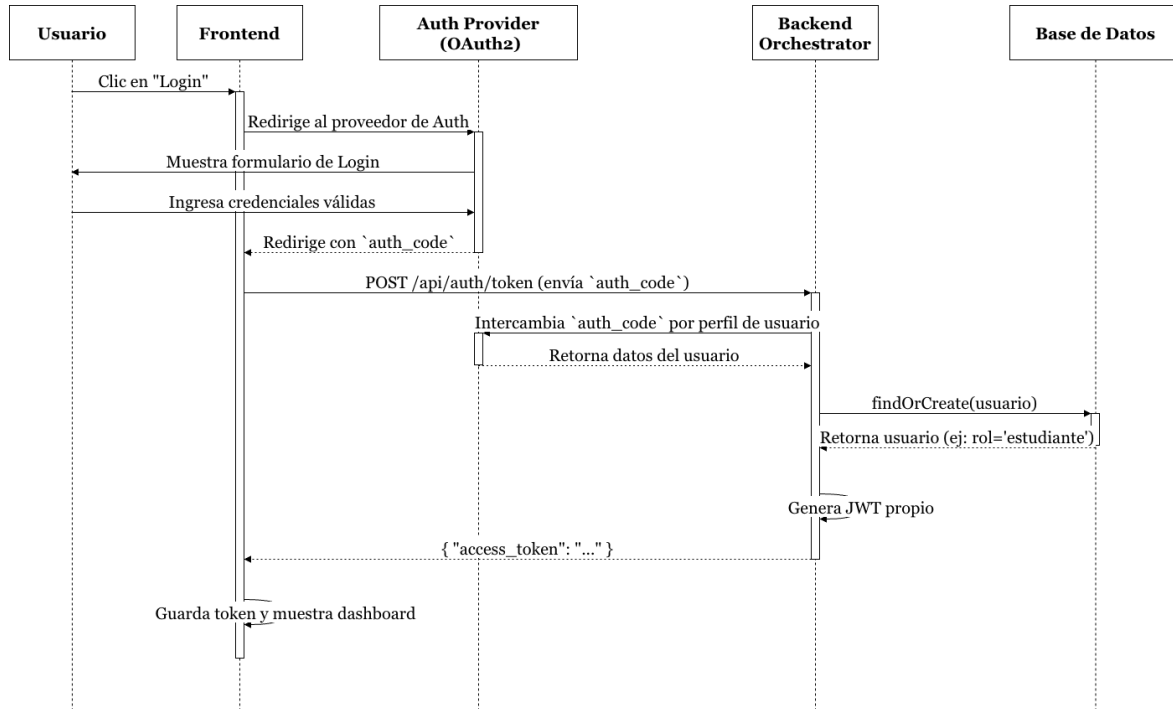


FIGURA 4: Diagrama de secuencia, Autenticación de usuario

Este diagrama en la Fig. 4 ilustra un flujo de autenticación estándar OIDC/OAuth2. El usuario es redirigido desde el frontend a un proveedor de autenticación externo. Al validarse con éxito, el proveedor retorna al frontend con un código, que se envía a la capa de orquestación. El backend intercambia este código por el perfil del usuario, consulta la base de datos para encontrarlo o crearlo, y finalmente genera un token JWT interno que devuelve al frontend para gestionar la sesión del usuario.

6.2. Reflexión vía texto

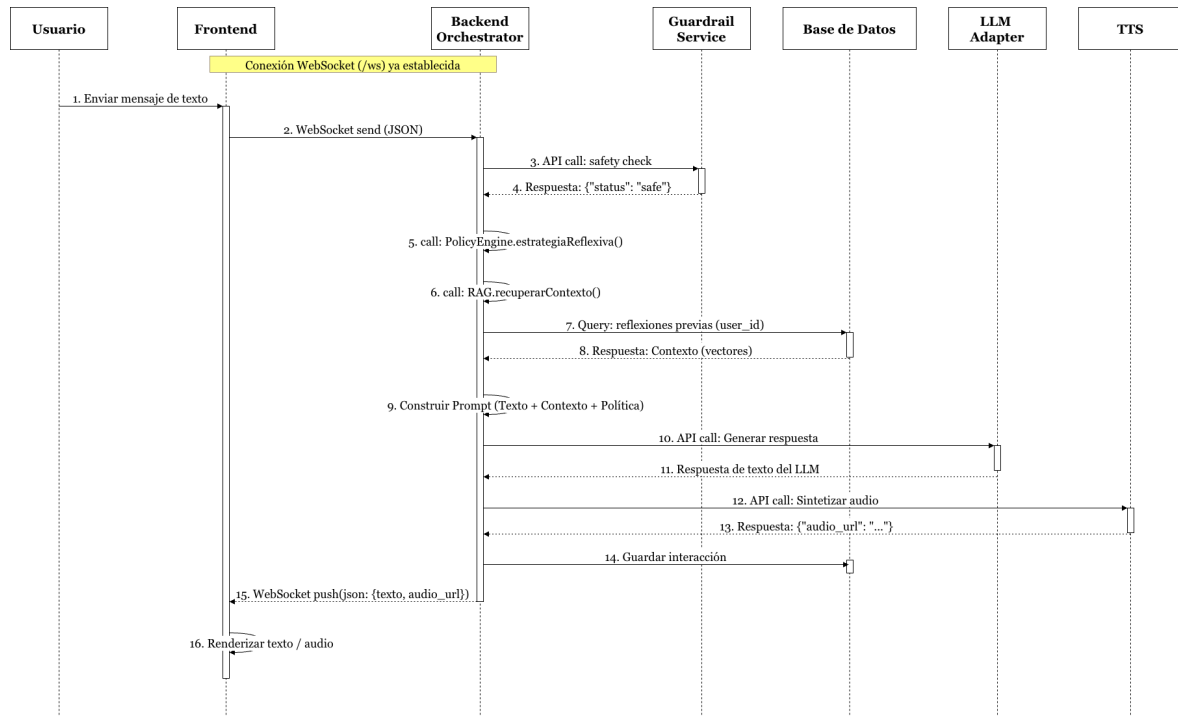


FIGURA 5: Diagrama de secuencia, Reflexión de un estudiante vía input de texto

Este es el bucle de conversación principal. Sobre una conexión websocket, el usuario envía un mensaje de texto. El backend recibe el mensaje y coordina la secuencia: primero, valida el contenido con el GuardRail, segundo, consulta la estrategia pedagógica del PolicyEngine, tercero, obtiene contexto via RAG (filtrado por usuario); y finalmente, genera una respuesta a través del LLM Adapter. Una vez generada, la respuesta de texto puede convertirse a audio, para ofrecer una mejor experiencia de usuario.

6.3. Creación de una actividad por un usuario docente

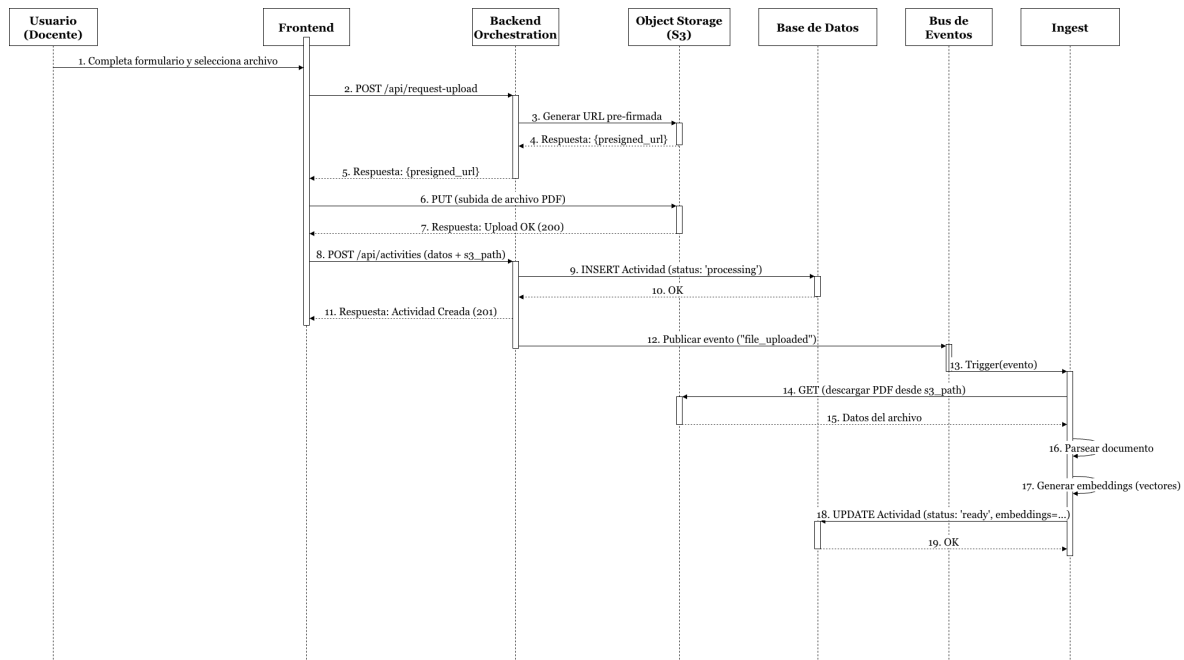


FIGURA 6: Diagrama de secuencia, creación de actividad por usuario docente

El diagrama de la Fig. 6 muestra el proceso de creación de una actividad por un docente, para incentivar el proceso metacognitivo en los estudiantes. Como parte del proceso, es posible subir un archivo que será utilizado como contexto para quienes realicen la actividad. Un pipeline desacoplado, se encarga de manejar la carga de archivos que luego serán procesados para generar embeddings para la base de datos.

6.4. Clasificación de la reflexión

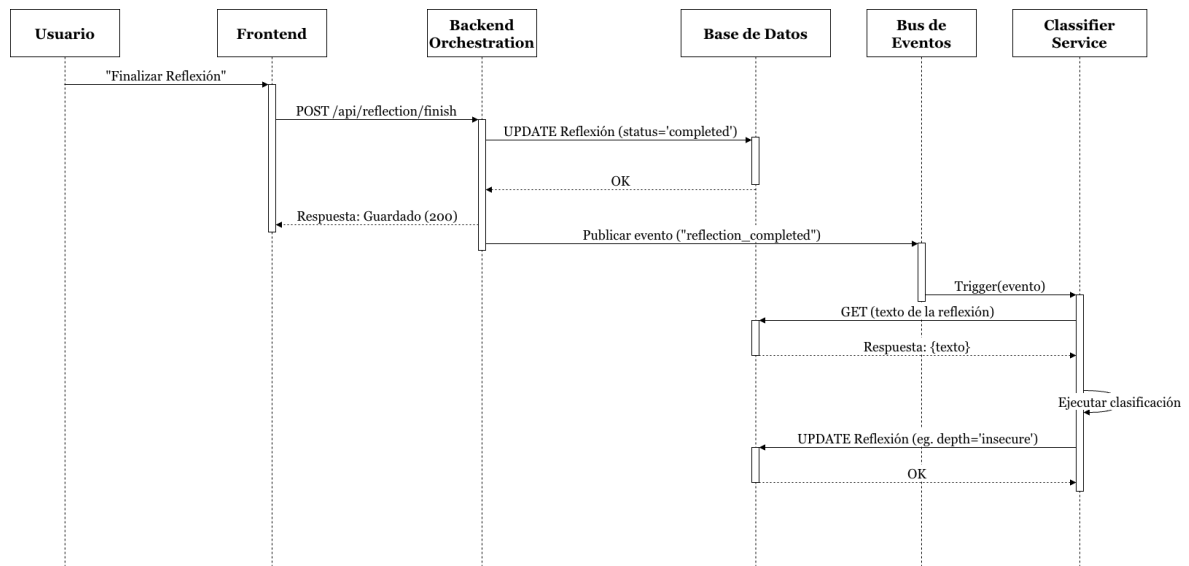


FIGURA 7: Diagrama de secuencia, clasificación de la reflexión

Cuando el usuario "Finaliza la Reflexión", el backend publica un evento en el bus de eventos para disparar la ejecución del servicio Classifier, que recupera el texto de la reflexión desde la base de datos, ejecuta el modelo de clasificación para determinar el sentimiento de la reflexión, y actualiza el registro con el resultado (ej. 'analítico', 'crítico'), todo de manera asincrónica.

7. Stack Tecnológico

7.1. Frontend

- Implementación: React, Typescript
- Despliegue/Infraestructura: Hosting estático (Amazon Web Services, Simple Storage Service - S3)

7.2. Backend

- Implementación: Typescript, ExpressJS
- Despliegue/Infraestructura: Amazon Web Services, Render (sujeto a decisión)
- LLM/TTS/STT Providers: OpenAI, Google/Gemini
- Procesamiento de archivos para embeddings: Docling

7.3. Base de datos

- Motor relacional: PostgreSQL
- Plugin vectorial: Pgvector

7.4. Control de versiones

- Bitbucket

8. Equipo de trabajo

Rol	Responsabilidades	Integrante
Project Manager / Backend Dev	Planificación, gestión de sprints, coordinación del equipo, arquitectura y desarrollo backend.	Nicolás Rossi
Product Owner / Frontend Dev	Definición de requerimientos, diseño funcional, experiencia de usuario (UX/UI), desarrollo de interfaz y priorización del backlog.	Santos Braun
Fullstack Dev / QA	Desarrollo integral de componentes (front y back), integración de servicios, pruebas unitarias y automatizadas.	Tomás Freire
Fullstack Dev / QA	Desarrollo integral de componentes (front y back), integración de servicios, pruebas unitarias y automatizadas.	Federico Inti García Lauberer

TABLA 1: Roles y responsabilidades del equipo de desarrollo de Milo

Glosario de términos técnicos

CDN *Content Delivery Network*: red de servidores distribuida que mejora el rendimiento de entrega de contenido al usuario. 13

Embedding Vector de representación que captura características semánticas de un objeto. 14

RAG *Retrieval-Augmented Generation*: técnica que combina recuperación de información con generación. 15

Serverless Modelo de computación en la nube donde la ejecución del código es gestionada por el proveedor de servicios, sin necesidad de administrar servidores. Permite escalar automáticamente y pagar solo por uso efectivo. 12, 14, 15

SPA *Single-Page Application*: aplicación web que carga una única página y actualiza dinámicamente su contenido. 12

Acrónimos

LLM Large Language Model. 14, 15

RAG Retrieval-Augmented Generation. 14, 15

S3 Simple Storage Service. 14

STT Speech-to-Text. 15

TTS Text-to-Speech. 15

Referencias

Cole Styker. (2024). *What is an AI stack?* [Recuperado de IBM]. <https://www.ibm.com/think/topics/ai-stack>

Gallup. (2023). *Ways to realign higher education with today's workforce* [Recuperado de Gallup]. <https://www.gallup.com/education/231740/ways-realign-higher-education-today-workforce.aspx>

Investopedia. (2024). *Essential job-hunting skills* [Recuperado de Investopedia]. <https://www.investopedia.com/essential-job-hunting-skills-8773801>

LinkedIn. (2024). *Global talent trends 2024* [Recuperado de LinkedIn]. <https://business.linkedin.com/talent-solutions/global-talent-trends>

Strauss, K. (2016, mayo). *These are the skills bosses say new college grads do not have* [Recuperado de Forbes]. Forbes. <https://www.forbes.com/sites/karstenstrauss/2016/05/17/these-are-the-skills-bosses-say-new-college-grads-do-not-have/>